

Collège de Maisonneuve

Développement de systèmes

Examen Final pt. 1

Émile Valade

Professeur : Jean-Pierre Fiset

Date de remise : 21 septembre 2026 à 16 h

Table des matières

1	Introduction	2
2	User Stories	2
2.1	User Story 1	2
2.2	User Story 2	2
2.3	User Story 3	2
3	MVP	2
3.1	MVP 1 – Page d’accueil	2
3.1.1	Description	2
3.1.2	Captures d’écran	3
3.1.3	Code source	3
3.2	MVP 2 – Catalogue	6
3.2.1	Description	6
3.2.2	Capture d’écran	7
3.2.3	Code source	7
3.3	MVP 3 – Recherche	9
3.3.1	Description	9
3.3.2	Capture d’écran	9
3.3.3	Code source	9
3.4	MVP 4 – Filtrage par catégorie	10
3.4.1	Description	10
3.4.2	Capture d’écran	11
3.4.3	Code source	11
3.5	MVP 5 – Page de détail du produit	12
3.5.1	Description	12
3.5.2	Capture d’écran	13
3.5.3	Code source	13
4	Inclusivité et accessibilité	15
4.1	Modification de la taille du texte	15
4.1.1	User Story	15
4.1.2	Description	15
4.1.3	Captures d’écran	16
4.1.4	Modifications et ajouts au code source	16
4.2	Thème clair et sombre	18
4.2.1	User Story	18
4.2.2	Description	18
4.2.3	Captures d’écran	18
4.2.4	Modifications et ajouts au code source	19
4.3	Changement de langue	21
4.3.1	User Story	21
4.3.2	Description	21
4.3.3	Captures d’écran	21
4.3.4	Modifications et ajouts au code source	22
5	Conclusion	25

1 Introduction

AirSmokes est une application Web permettant d'acheter des produits de tabagie, développée avec React, TypeScript et Next.js. L'interface utilise des composants communs pour la navigation et la mise en page.

Le MVP actuel ne comporte pas de base de données et ne permet pas de passer de réelles commandes. Pour l'instant, il s'agit davantage d'un catalogue que d'un véritable marché en ligne.

2 User Stories

2.1 User Story 1

En tant que client, je souhaite parcourir le catalogue de produits afin de pouvoir consulter les produits disponibles.

2.2 User Story 2

En tant que client, je souhaite rechercher un produit afin de trouver rapidement le produit qui m'intéresse.

2.3 User Story 3

En tant que client, je souhaite filtrer les produits par catégorie afin de pouvoir consulter uniquement les produits correspondant à mes besoins.

3 MVP

3.1 MVP 1 – Page d'accueil

3.1.1 Description

La page d'accueil constitue le point d'entrée principal du site. Elle permet à l'utilisateur de découvrir le site, d'accéder rapidement au catalogue et de parcourir les différentes catégories de produits disponibles.

3.1.2 Captures d'écran

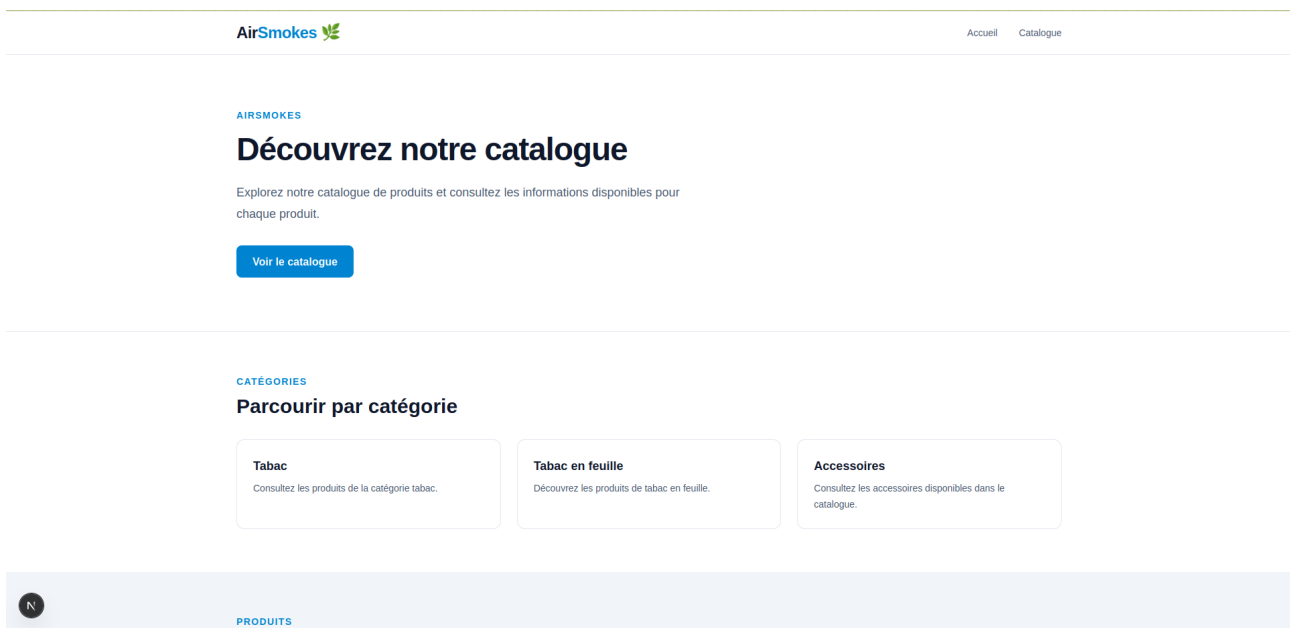


FIGURE 1 – Écran d'accueil – partie 1

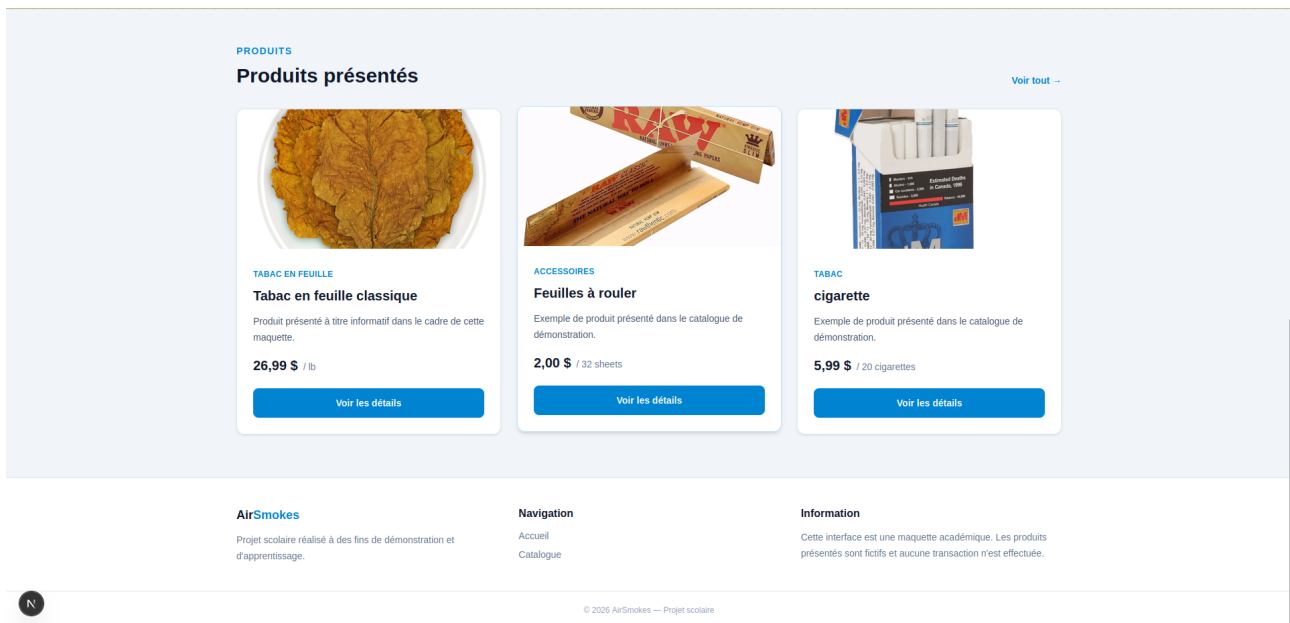


FIGURE 2 – Écran d'accueil – partie 2

3.1.3 Code source

```
1 import Link from "next/link";
2 import ProductCard from "@app/components/ProductCard";
3 import { products } from "@app/data/product";
4
5 export default function Home() {
6   return ( <div>
```

```

7  { /* Hero */ } <section className="border-b border-slate-200 bg-white dark:
    border-slate-800 dark:bg-slate-900"> <div className="mx-auto max-w-7xl
        px-6 py-20"> <div className="max-w-3xl"> <p className="text-sm font-
            semibold uppercase tracking-widest text-sky-600 dark:text-sky-400">
8  AirSmokes </p>
9
10  ““
11      <h1 className="mt-4 text-5xl font-bold tracking-tight text-slate-900
            dark:text-white">
12          D couvrez notre catalogue
13      </h1>
14
15      <p className="mt-6 max-w-2xl text-lg leading-8 text-slate-600 dark:
            text-slate-400">
16          Explorez notre catalogue de produits et consultez les informations
17          disponibles pour chaque produit.
18      </p>
19
20      <div className="mt-8">
21          <Link
22              href="/catalogue"
23              className="inline-block rounded-lg bg-sky-600 px-6 py-3 font-
                semibold text-white transition hover:bg-sky-700"
24          >
25              Voir le catalogue
26          </Link>
27      </div>
28  </div>
29 </div>
30 ““
31
32 </section>
33
34 { /* Categories */ }
35
36 <section className="mx-auto max-w-7xl px-6 py-16">
37     <div className="mb-8">
38         <p className="text-sm font-semibold uppercase tracking-widest text-
            sky-600 dark:text-sky-400">
39             Cat gories
40         </p>
41
42     ““
43     <h2 className="mt-2 text-3xl font-bold text-slate-900 dark:text-white">
44         Parcourir par cat gorie
45     </h2>
46 </div>
47
48 <div className="grid gap-6 md:grid-cols-3">
49     <Link
50         href="/catalogue?category=tabac"
51         className="rounded-xl border border-slate-200 bg-white p-6 transition
            hover:border-sky-300 hover:shadow-md dark:border-slate-800 dark:

```

```

    bg-slate-900 dark: hover: border-sky-700"
52 >
53 <h3 className="text-lg font-bold text-slate-900 dark:text-white">
54     Tabac
55 </h3>
56
57 <p className="mt-2 text-sm leading-6 text-slate-600 dark:text-slate
    -400">
58     Consultez les produits de la cat gorie tabac.
59 </p>
60 </Link>
61
62 <Link
63     href="/catalogue?category=tabac%20en%20feuille"
64     className="rounded-xl border border-slate-200 bg-white p-6 transition
        hover: border-sky-300 hover: shadow-md dark: border-slate-800 dark:
        bg-slate-900 dark: hover: border-sky-700"
65 >
66 <h3 className="text-lg font-bold text-slate-900 dark:text-white">
67     Tabac en feuille
68 </h3>
69
70 <p className="mt-2 text-sm leading-6 text-slate-600 dark:text-slate
    -400">
71     D couvrez les produits de tabac en feuille.
72 </p>
73 </Link>
74
75 <Link
76     href="/catalogue?category=accessoires"
77     className="rounded-xl border border-slate-200 bg-white p-6 transition
        hover: border-sky-300 hover: shadow-md dark: border-slate-800 dark:
        bg-slate-900 dark: hover: border-sky-700"
78 >
79 <h3 className="text-lg font-bold text-slate-900 dark:text-white">
80     Accessoires
81 </h3>
82
83 <p className="mt-2 text-sm leading-6 text-slate-600 dark:text-slate
    -400">
84     Consultez les accessoires disponibles dans le catalogue.
85 </p>
86 </Link>
87 </div>
88 ""
89
90 </section>
91
92 { /* Featured products */ }
93
94 <section className="bg-slate-100 dark: bg-slate-950">
95     <div className="mx-auto max-w-7xl px-6 py-16">
96         <div className="mb-8 flex items-end justify-between gap-4">

```

```

97     <div>
98         <p className="text-sm font-semibold uppercase tracking-widest
99             text-sky-600 dark:text-sky-400">
100             Produits
101         </p>
102     “ “
103         <h2 className="mt-2 text-3xl font-bold text-slate-900 dark:text-
104             white">
105             Produits pr sent s
106         </h2>
107     </div>
108     <Link
109         href="/catalogue"
110         className="hidden text-sm font-semibold text-sky-600 hover:text-sky-
111             700 sm:block dark:text-sky-400 dark: hover: text-sky-300"
112     >
113         Voir tout
114     </Link>
115 </div>
116 <div className="grid gap-6 md:grid-cols-3">
117     {products.slice(0, 3).map((product) => (
118         <ProductCard key={product.id} product={product} />
119     ))}
120 </div>
121 </div>
122 “ “
123
124 </section>
125 </div>
126 );
127 }

```

3.2 MVP 2 – Catalogue

3.2.1 Description

Le catalogue permet à l'utilisateur de consulter les différents produits disponibles sur le site. Les produits sont présentés de manière structurée afin de faciliter leur consultation.

3.2.2 Capture d'écran

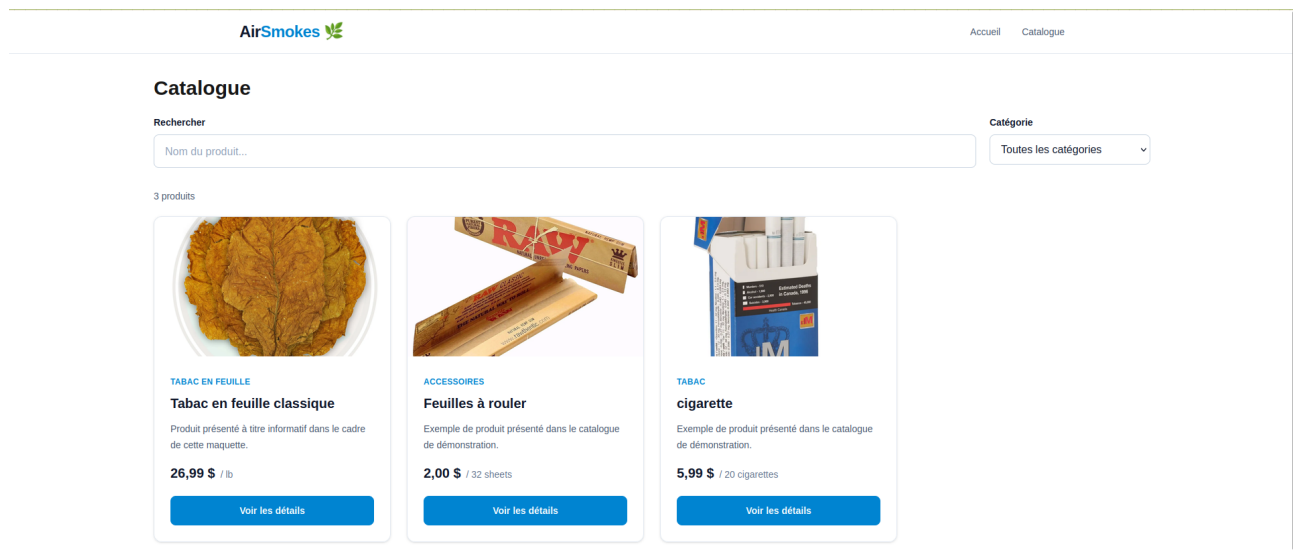


FIGURE 3 – Écran du catalogue

3.2.3 Code source

```
1 import SearchBar from "@app/components/SearchBar";
2 import ProductCard from "@app/components/ProductCard";
3 import CategoryFilter from "@app/components/CategoryFilter";
4 import { products } from "@app/data/product";
5
6 type CataloguePageProps = {
7   searchParams: Promise<{
8     q?: string;
9     category?: string;
10   }>;
11 };
12
13 export default async function CataloguePage({
14   searchParams,
15 }: CataloguePageProps) {
16   const params = await searchParams;
17
18   const query = params.q?.toLowerCase().trim() ?? "";
19   const category = params.category?.toLowerCase().trim() ?? "";
20
21   const filteredProducts = products.filter((product) => {
22     const searchText = `
23       ${product.name}
24       ${product.description}
25       ${product.category}
26     `.toLowerCase();
27
28     const matchesSearch = searchText.includes(query);
29
30     const matchesCategory =
```



```

30 !category ||
31 category === "all" ||
32 product.category.toLowerCase() === category;
33
34 return matchesSearch && matchesCategory;
35 });
36
37 return ( <main className="container mx-auto px-6 py-8"> <h1 className="mb
    -6 text-3xl font-bold">
38 Catalogue </h1>
39
40   <div className="mb-8">
41     <div className="grid gap-5 md:grid-cols-[1fr_240px]">
42       <SearchBar />
43       <CategoryFilter />
44     </div>
45   </div>
46
47   <div className="mb-5">
48     <p className="text-sm text-slate-600 dark:text-slate-400">
49       {filteredProducts.length} produit
50       {filteredProducts.length !== 1 ? "s" : ""}
51     </p>
52   </div>
53
54   {filteredProducts.length > 0 ? ( <div className="grid grid-cols-1 gap-6
    sm:grid-cols-2 lg:grid-cols-4">
55     {filteredProducts.map((product) => ( <ProductCard key={product.id}
    product={product} />
56   ))} </div>
57 ) : ( <div className="py-16 text-center"> <p className="text-lg font-
    medium">
58 Aucun produit trouv </p>
59
60   “ “
61   {query && (
62     <p className="mt-2 text-sm text-base-content/60">
63       Aucun r sultat pour {query}
64     </p>
65   )}
66 </div>
67   “ “
68
69 )}
70
71 </main>
72 );
73 }

```

3.3 MVP 3 – Recherche

3.3.1 Description

L'écran du catalogue intègre une barre de recherche permettant à l'utilisateur de rechercher des produits dynamiquement. Les résultats sont mis à jour au fur et à mesure que l'utilisateur saisit sa recherche.

3.3.2 Capture d'écran

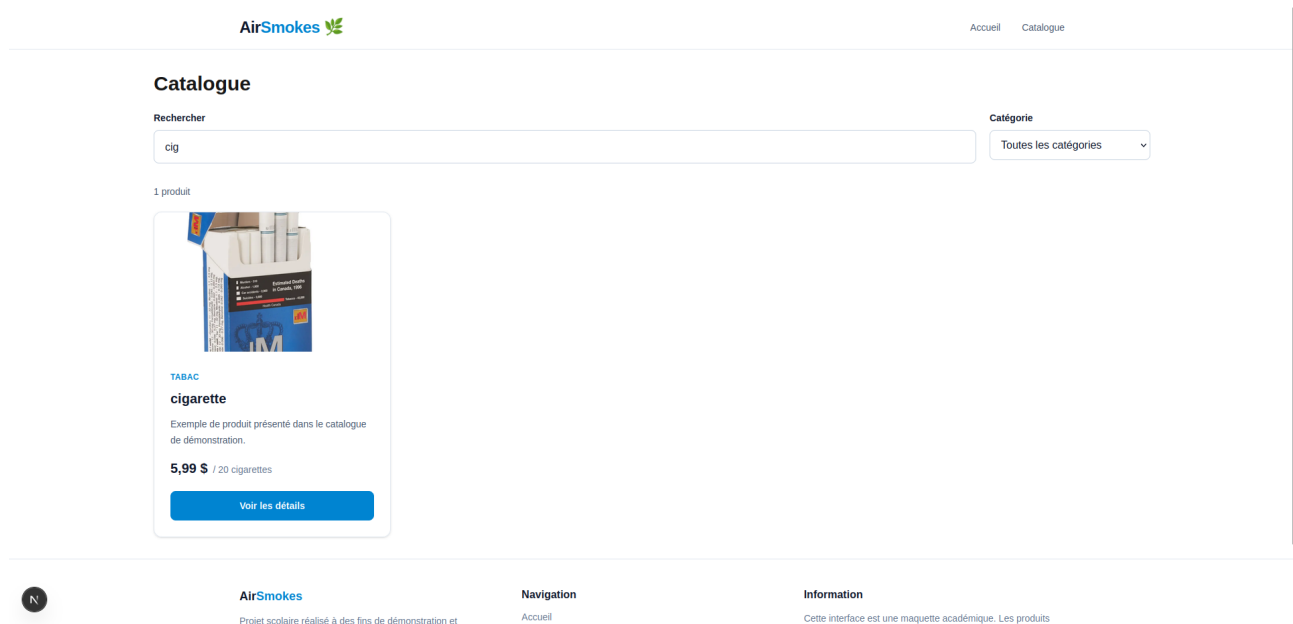


FIGURE 4 – Recherche dynamique dans le catalogue

3.3.3 Code source

```
1 "use client";
2
3 import { useRouter, useSearchParams } from "next/navigation";
4
5 export default function SearchBar() {
6   const router = useRouter();
7   const searchParams = useSearchParams();
8
9   function handleSearch(value: string) {
10    const params = new URLSearchParams(searchParams.toString());
11
12    if (value.trim()) {
13      params.set("q", value);
14    } else {
15      params.delete("q");
16    }
17
18    router.push(`/catalogue?${params.toString()}`);
19  }
20
21  return ( <div> <label
```

```

22 | htmlFor="search"
23 | className="mb-2 block text-sm font-semibold text-slate-900 dark:text-
    | white"
24 |
25 | >
26 |
27 | Rechercher </label>
28 |
29 | <input
30 | id="search"
31 | type="search"
32 | placeholder="Nom du produit..."
33 | defaultValue={searchParams.get("q") ?? ""}
34 | onChange={(e) => handleSearch(e.target.value)}
35 | className="w-full rounded-lg border border-slate-300 bg-white px-4 py-3
    | text-slate-900 outline-none transition placeholder:text-slate-400
    | focus:border-sky-500 focus:ring-2 focus:ring-sky-100 dark:border-slate
    | -700 dark:bg-slate-950 dark:text-white dark:focus:border-sky-500 dark:
    | focus:ring-sky-950"
36 | />
37 |
38 | </div>
39 | );
40 | }

```

3.4 MVP 4 – Filtrage par catégorie

3.4.1 Description

L'écran du catalogue permet à l'utilisateur de filtrer les produits selon leur catégorie. La sélection d'une catégorie met à jour les produits affichés afin de ne présenter que ceux correspondant au filtre sélectionné.

3.4.2 Capture d'écran

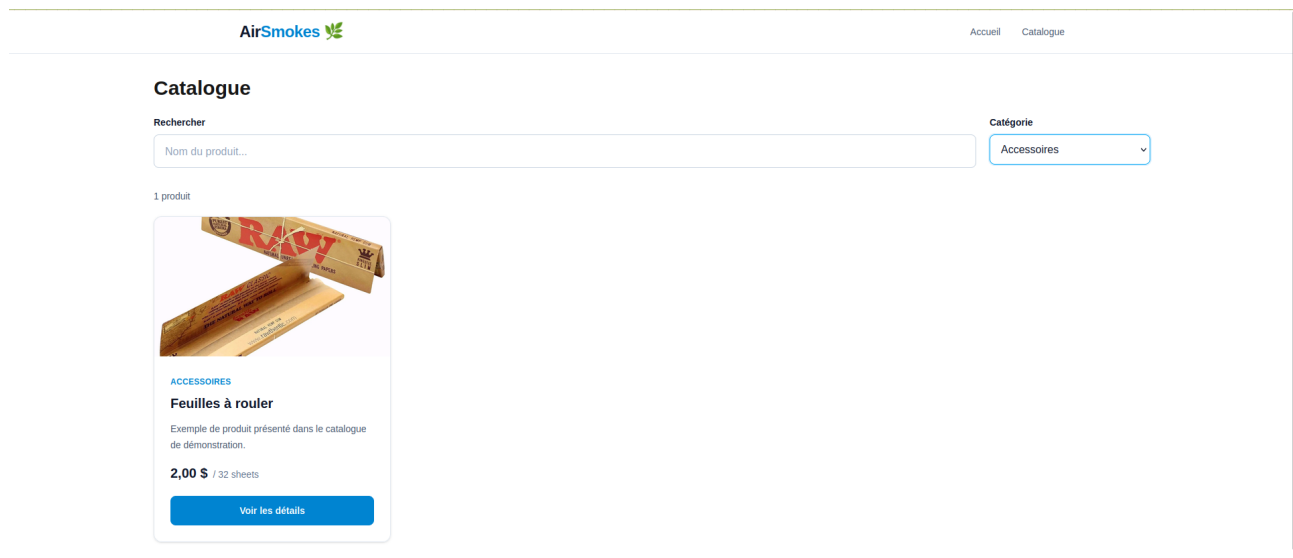


FIGURE 5 – Filtrage des produits par catégorie

3.4.3 Code source

```
1  "use client";
2
3  import { useRouter, useSearchParams } from "next/navigation";
4
5  export default function CategoryFilter() {
6    const router = useRouter();
7    const searchParams = useSearchParams();
8
9    function handleCategoryChange(value: string) {
10     const params = new URLSearchParams(searchParams.toString());
11
12     if (value && value !== "all") {
13       params.set("category", value);
14     } else {
15       params.delete("category");
16     }
17
18     router.push(`/catalogue?${params.toString()}`);
19   }
20
21   return ( <div> <label
22     htmlFor="category"
23     className="mb-2 block text-sm font-semibold text-slate-900 dark:text-
24       white"
25   >
26
27     Cat gorie </label>
28
```

```

29 <select
30 id="category"
31 value={searchParams.get("category") ?? "all"}
32 onChange={(e) => handleCategoryChange(e.target.value)}
33 className="w-full rounded-lg border border-slate-300 bg-white px-4 py-3
    text-slate-900 outline-none transition focus:border-sky-500 focus:ring
    -2 focus:ring-sky-100 dark:border-slate-700 dark:bg-slate-950 dark:
    text-white dark:focus:border-sky-500 dark:focus:ring-sky-950"
34
35 >
36
37 ‘ ‘ ‘
38 <option value="all">Toutes les cat gories </option>
39 ‘ ‘ ‘
40
41 ‘ ‘ ‘
42 <option value="tabac">Tabac</option>
43 <option value="tabac en feuille">Tabac en feuille</option>
44 <option value="accessoires">Accessoires</option>
45 ‘ ‘ ‘
46
47 </select>
48 </div>
49 );
50 }

```

3.5 MVP 5 – Page de détail du produit

3.5.1 Description

Lorsqu’un utilisateur sélectionne un produit dans le catalogue, il est redirigé vers une page de détail présentant les informations supplémentaires associées au produit.

3.5.2 Capture d'écran

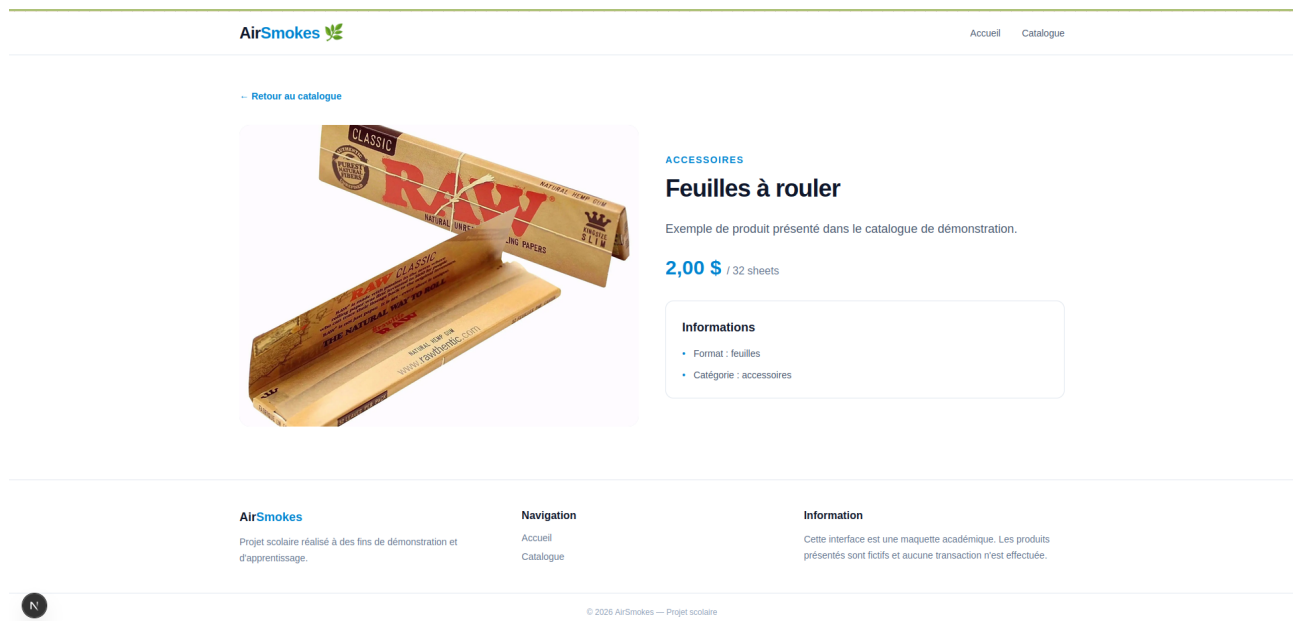


FIGURE 6 – Écran de détail du produit

3.5.3 Code source

```
1 import Image from "next/image";
2 import Link from "next/link";
3 import { notFound } from "next/navigation";
4 import { products } from "@app/data/product";
5
6 type ProductPageProps = {
7   params: Promise<{
8     id: string;
9   }>;
10 };
11
12 export default async function ProductPage({
13   params,
14 }: ProductPageProps) {
15   const { id } = await params;
16
17   const product = products.find((product) => product.id === id);
18
19   if (!product) {
20     notFound();
21   }
22
23   const formattedPrice = new Intl.NumberFormat("fr-CA", {
24     style: "currency",
25     currency: "CAD",
26   }).format(product.price);
27
28   return ( <div className="mx-auto max-w-7xl px-6 py-12"> <Link
29     href="/catalogue"
```

```

30   className="text-sm font-semibold text-sky-600 hover:text-sky-700 dark:
      text-sky-400 dark: hover:text-sky-300"
31
32   >
33
34       Retour au catalogue </Link>
35
36   <div className="mt-8 grid gap-10 lg:grid-cols-2">
37
38       ““
39   <div className="relative min-h-[450px] overflow-hidden rounded-2xl bg-
      slate-200 dark:bg-slate-800">
40       <Image
41           src={product.image}
42           alt={product.name}
43           fill
44           className="object-cover"
45           priority
46       />
47   </div>
48
49   <div className="flex flex-col justify-center">
50       <span className="text-sm font-semibold uppercase tracking-widest text-
          sky-600 dark:text-sky-400">
51           {product.category}
52       </span>
53
54       <h1 className="mt-3 text-4xl font-bold tracking-tight text-slate-900
          dark:text-white">
55           {product.name}
56       </h1>
57
58       <p className="mt-6 text-lg leading-8 text-slate-600 dark:text-slate-400
          ">
59           {product.description}
60       </p>
61
62       <div className="mt-6 flex items-baseline gap-2">
63           <span className="text-3xl font-bold text-sky-600 dark:text-sky-400">
64               {formattedPrice}
65           </span>
66
67           <span className="text-base text-slate-500 dark:text-slate-400">
68               / {product.qty}
69           </span>
70       </div>
71
72   <div className="mt-8 rounded-xl border border-slate-200 bg-white p-6
      dark:border-slate-800 dark:bg-slate-900">
73       <h2 className="text-lg font-bold text-slate-900 dark:text-white">
74           Informations
75       </h2>
76

```

```

77     <ul className="mt-4 space-y-3">
78         {product.details.map((detail) => (
79             <li
80                 key={detail}
81                 className="flex gap-3 text-sm text-slate-600 dark:text-slate
                    -400"
82             >
83                 <span className="text-sky-600 dark:text-sky-400">
84
85                     </span>
86                     {detail}
87                 </li>
88             )})}
89     </ul>
90 </div>
91 </div>
92 ‘‘‘
93
94     </div>
95 </div>
96 );
97 }

```

4 Inclusivité et accessibilité

L'application prend en considération différents besoins d'accessibilité et d'inclusivité afin de rendre son utilisation plus simple pour un plus grand nombre d'utilisateurs.

Plusieurs fonctionnalités d'accessibilité sont intégrées à l'application, notamment des options permettant de modifier la taille du texte, de choisir un thème clair ou sombre et de changer la langue de l'interface afin de faciliter la lecture et l'utilisation de l'application.

4.1 Modification de la taille du texte

4.1.1 User Story

En tant qu'utilisateur ayant des difficultés à lire du texte de petite taille, je souhaite pouvoir augmenter la taille du texte affiché sur le site afin de faciliter la lecture et la consultation des informations.

4.1.2 Description

Une fonctionnalité permettant de modifier la taille du texte a été ajoutée à l'application afin d'améliorer la visibilité et l'accessibilité de l'interface.

L'utilisateur peut augmenter ou diminuer la taille du texte à l'aide d'un contrôle accessible depuis l'interface. La modification de la taille est appliquée aux différents éléments textuels de l'application afin de rendre le contenu plus facile à lire.

Cette fonctionnalité permet notamment aux utilisateurs ayant des difficultés à lire les textes de petite taille d'adapter l'interface à leurs besoins visuels.

4.1.3 Captures d'écran

Les captures d'écran ci-dessous montrent l'interface avant et après l'activation de l'option permettant d'augmenter la taille du texte.

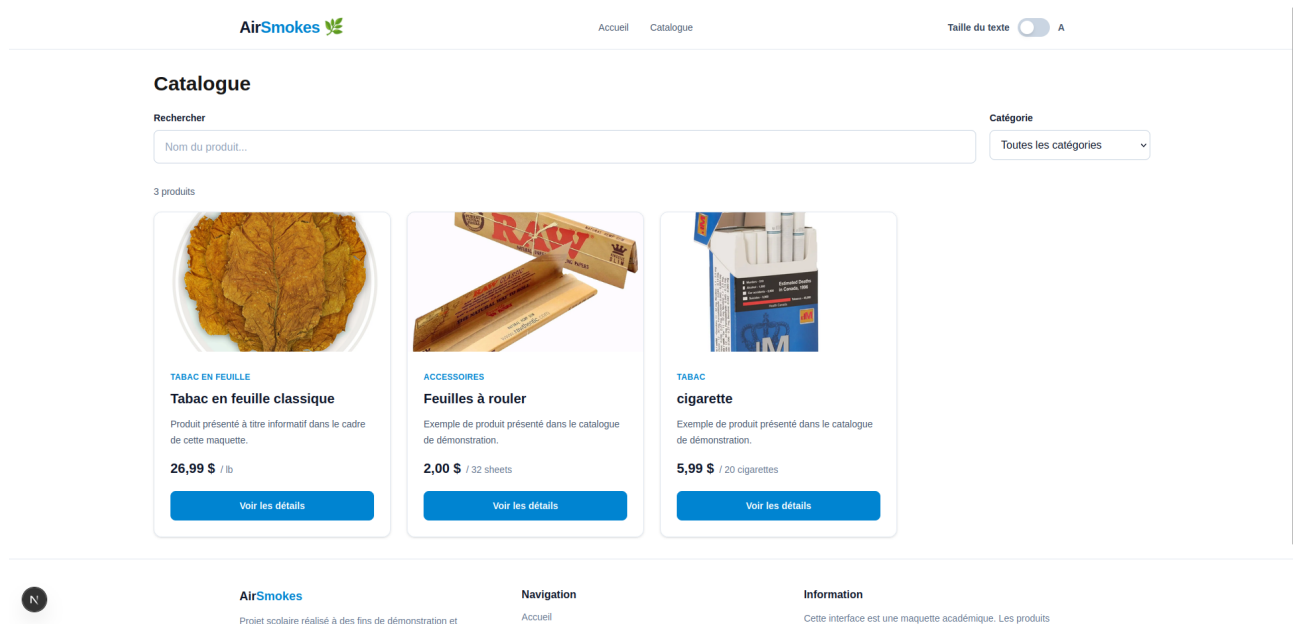


FIGURE 7 – Interface avec la taille de texte par défaut

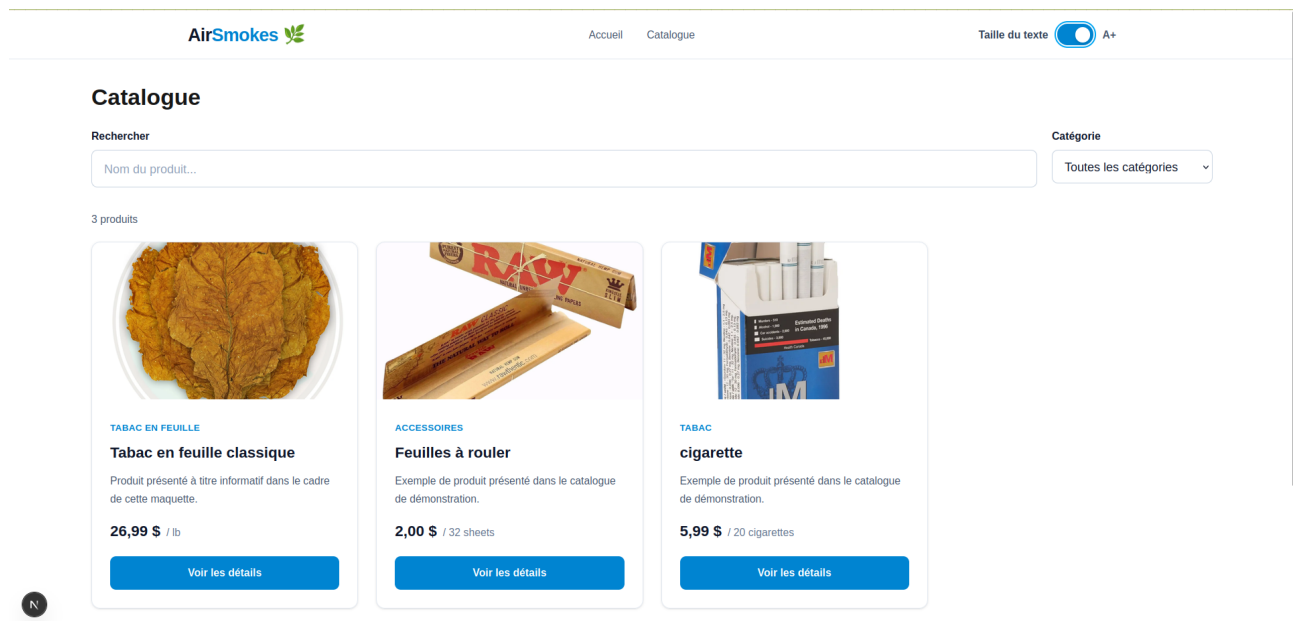


FIGURE 8 – Interface avec une taille de texte augmentée

4.1.4 Modifications et ajouts au code source

La fonctionnalité repose sur un état permettant de déterminer la taille de texte actuellement sélectionnée. Cette valeur est ensuite utilisée pour appliquer la taille correspondante aux éléments de l'interface.

Lorsque l'utilisateur active l'option, la classe correspondant à une taille de texte plus grande est ajoutée à l'élément racine de la page. La préférence est également conservée dans le stockage local du navigateur afin de maintenir le choix de l'utilisateur lors de ses prochaines visites.

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  export default function FontSizeToggle() {
6    const [largeText, setLargeText] = useState(() => {
7      if (typeof window === "undefined") {
8        return false;
9      }
10
11      return localStorage.getItem("large-text") === "true";
12    });
13
14    useEffect(() => {
15      if (largeText) {
16        document.documentElement.classList.add("large-text");
17      } else {
18        document.documentElement.classList.remove("large-text");
19      }
20
21      localStorage.setItem("large-text", String(largeText));
22    }, [largeText]);
23
24    function toggleLargeText() {
25      setLargeText((current) => !current);
26    }
27
28    return ( <div className="flex items-center gap-3"> <span className="text-
      sm font-semibold text-slate-700 dark:text-slate-300">
29      Taille du texte </span>
30
31      <button
32        type="button"
33        role="switch"
34        aria-checked={largeText}
35        aria-label="Activer le texte plus grand"
36        onClick={toggleLargeText}
37        className={`relative h-7 w-12 shrink-0 rounded-full transition-colors
          duration-200 focus:outline-none focus:ring-2 focus:ring-sky-500 focus:
          ring-offset-2 dark:focus:ring-offset-slate-900 ${
38          largeText ? "bg-sky-600" : "bg-slate-300 dark:bg-slate-700"
39        }`}
40
41      >
42
43        “ “
44        <span
45          “ “
46
47          “ “
48          className={`absolute left-1 top-1 h-5 w-5 rounded-full bg-white shadow-
            md transition-transform duration-200 ${
49            largeText ? "translate-x-5" : ""

```

```

50     }‘}
51 />
52 ‘‘‘
53
54     </button>
55
56 <span
57 className="shrink-0 text-sm font-bold text-slate-600 dark:text-slate-400"
58 aria-hidden="true"
59
60 >
61
62     ‘‘‘
63 {largeText ? "A+" : "A"}
64 ‘‘‘
65
66     </span>
67 </div>
68 );
69 }

```

4.2 Thème clair et sombre

4.2.1 User Story

En tant qu'utilisateur ayant des préférences ou des besoins visuels différents, je souhaite pouvoir choisir entre un thème clair et un thème sombre afin de consulter l'application dans un environnement visuel qui me convient.

4.2.2 Description

Une fonctionnalité permettant de basculer entre un thème clair et un thème sombre a été ajoutée à l'application afin d'améliorer l'accessibilité et le confort visuel des utilisateurs.

L'utilisateur peut modifier l'apparence de l'interface à l'aide d'un bouton permettant d'activer ou de désactiver le thème sombre. Le changement est appliqué à l'ensemble de l'interface afin de conserver une apparence cohérente lors de la navigation sur le site.

Le thème clair permet une utilisation dans des conditions d'éclairage normales, tandis que le thème sombre peut notamment faciliter la consultation de l'application dans un environnement peu éclairé.

4.2.3 Captures d'écran

Les captures d'écran ci-dessous montrent l'interface avec les deux thèmes disponibles.

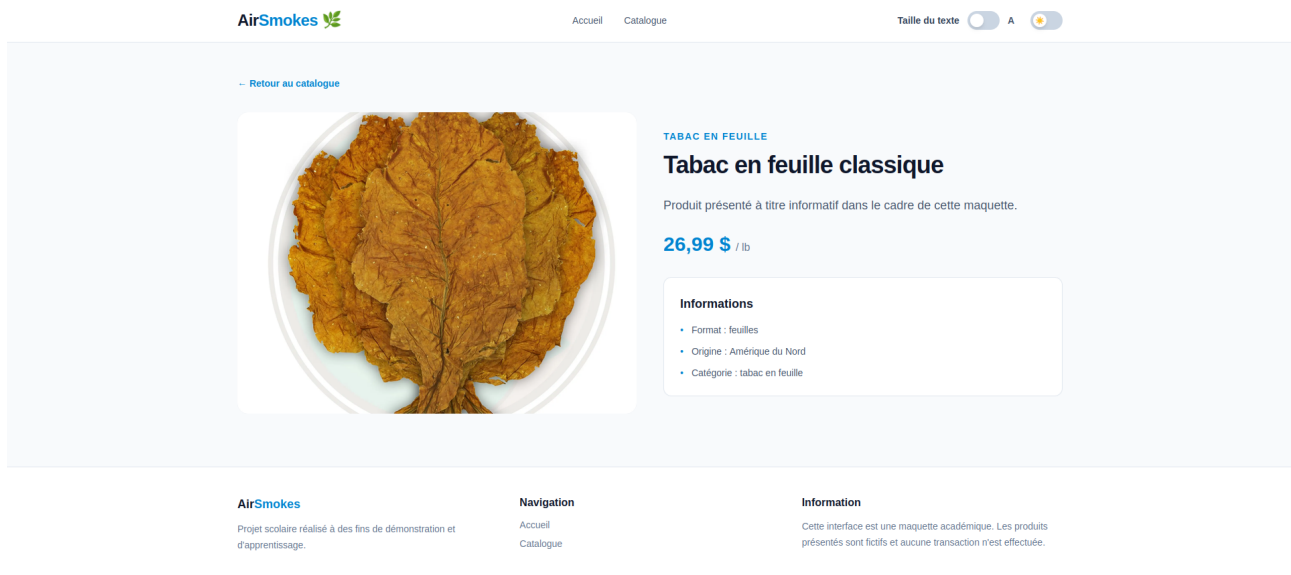


FIGURE 9 – Interface avec le thème clair

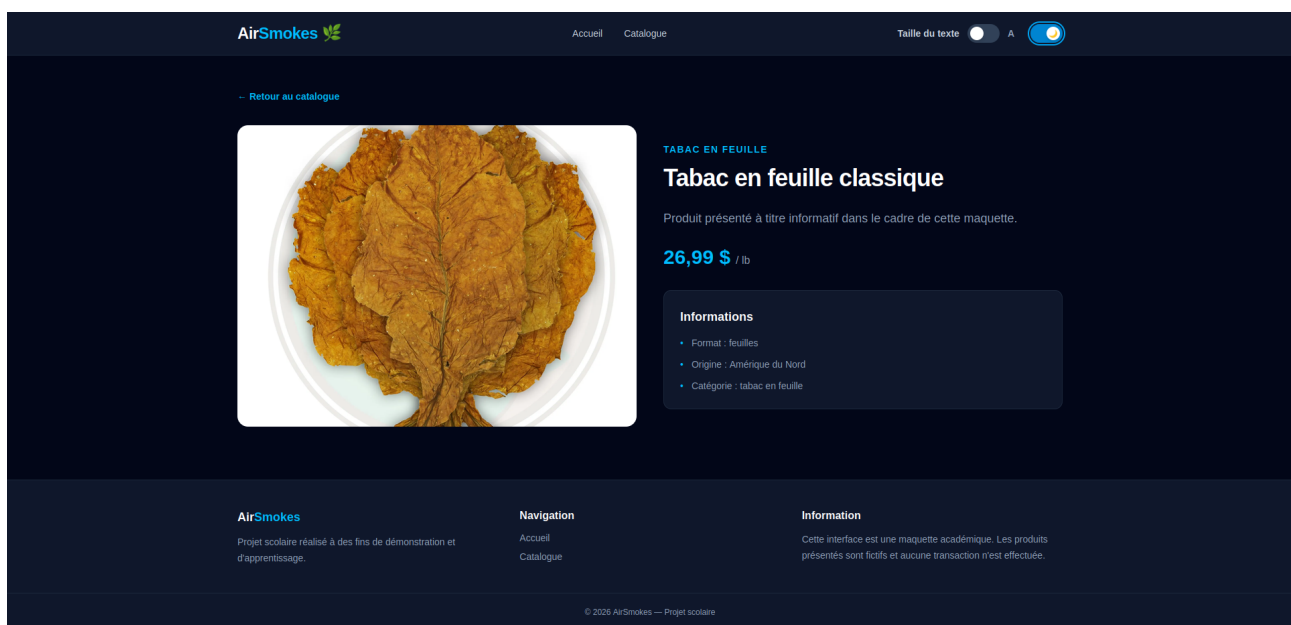


FIGURE 10 – Interface avec le thème sombre

4.2.4 Modifications et ajouts au code source

La fonctionnalité repose sur un contrôle permettant de modifier le thème actuellement utilisé par l'application. Le thème sélectionné est appliqué à l'interface afin de modifier les couleurs de fond, de texte et des différents composants.

Le style de l'interface est ensuite adapté en fonction du thème sélectionné afin de conserver un contraste suffisant et une présentation cohérente dans les deux modes.

```

1 "use client";
2
3 import { useEffect, useState } from "react";
4
5 export default function ThemeToggle() {

```

```

6  const [darkMode, setDarkMode] = useState(() => {
7  if (typeof window === "undefined") {
8  return false;
9  }
10
11  “““
12  return localStorage.getItem("dark-mode") === "true";
13  “““
14
15  });
16
17  useEffect(( ) => {
18  if (darkMode) {
19  document.documentElement.classList.add("dark");
20  } else {
21  document.documentElement.classList.remove("dark");
22  }
23
24  “““
25  localStorage.setItem("dark-mode", String(darkMode));
26  “““
27
28  }, [darkMode]);
29
30  function toggleTheme() {
31  setDarkMode((current) => !current);
32  }
33
34  return ( <div className="flex items-center gap-3">
35  <button
36  type="button"
37  role="switch"
38  aria-checked={darkMode}
39  aria-label="Activer le th me sombre"
40  onClick={toggleTheme}
41  className={‘relative h-[28px] w-[48px] shrink-0 rounded-full transition-
      colors duration-200 focus:outline-none focus:ring-2 focus:ring-sky-500
      focus:ring-offset-2 dark:focus:ring-offset-slate-900 ${
42      darkMode ? "bg-sky-600" : "bg-slate-300"
43      }‘}
44  >
45  <span
46  className={‘absolute left-[4px] top-[4px] flex h-[20px] w-[20px] items-
      center justify-center rounded-full bg-white shadow-md transition-
      transform duration-200 ${
47      darkMode ? "translate-x-[20px]" : "translate-x-0"
48      }‘}
49  > <span className="text-[11px]" aria-hidden="true">
50  {darkMode ? "      " : "      "} </span> </span> </button> </div>
51  );
52  }

```

4.3 Changement de langue

4.3.1 User Story

En tant qu'utilisateur préférant utiliser l'application dans une langue différente, je souhaite pouvoir choisir entre le français, l'anglais et l'espagnol afin de consulter le contenu de l'application dans la langue qui me convient.

4.3.2 Description

Une fonctionnalité permettant de changer la langue de l'application a été ajoutée afin de rendre l'interface plus accessible à un plus grand nombre d'utilisateurs.

L'utilisateur peut sélectionner la langue de son choix à l'aide d'un contrôle accessible depuis l'interface. Le contenu textuel de l'application est ensuite affiché dans la langue sélectionnée.

Cette fonctionnalité permet notamment aux utilisateurs francophones, anglophones et hispanophones d'utiliser l'application sans être limités par la langue de l'interface. Elle contribue également à rendre l'application plus inclusive dans un contexte multilingue.

4.3.3 Captures d'écran

Les captures d'écran ci-dessous montrent l'interface dans les trois langues disponibles.

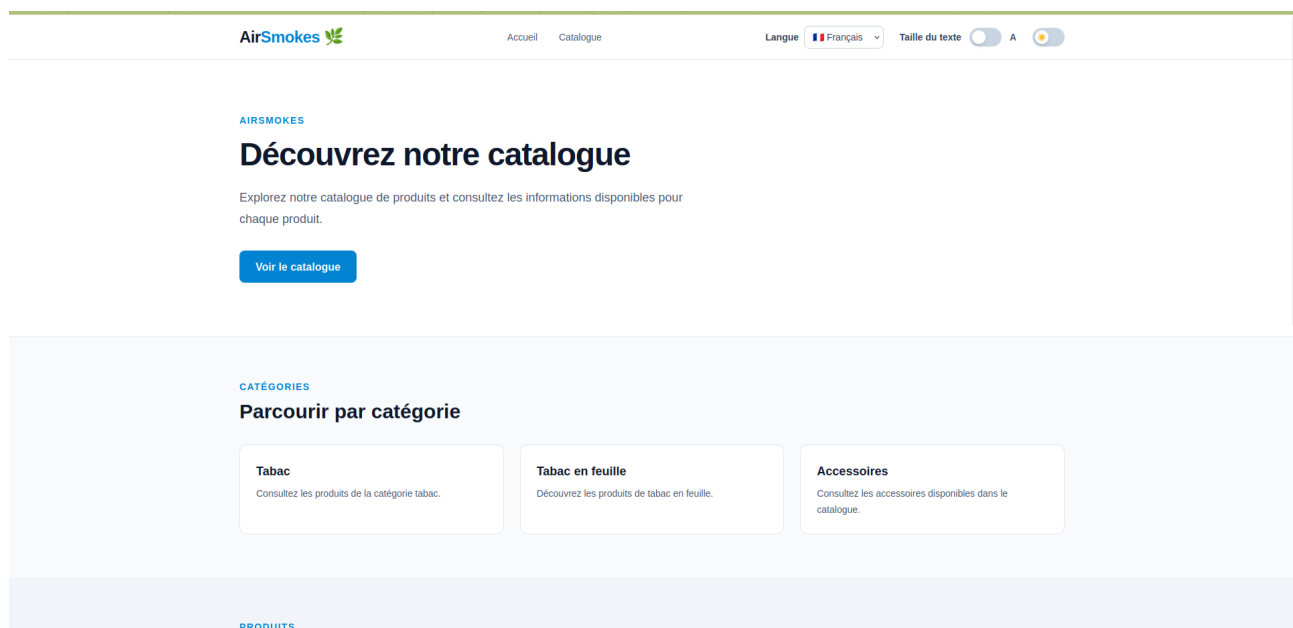


FIGURE 11 – Interface en français

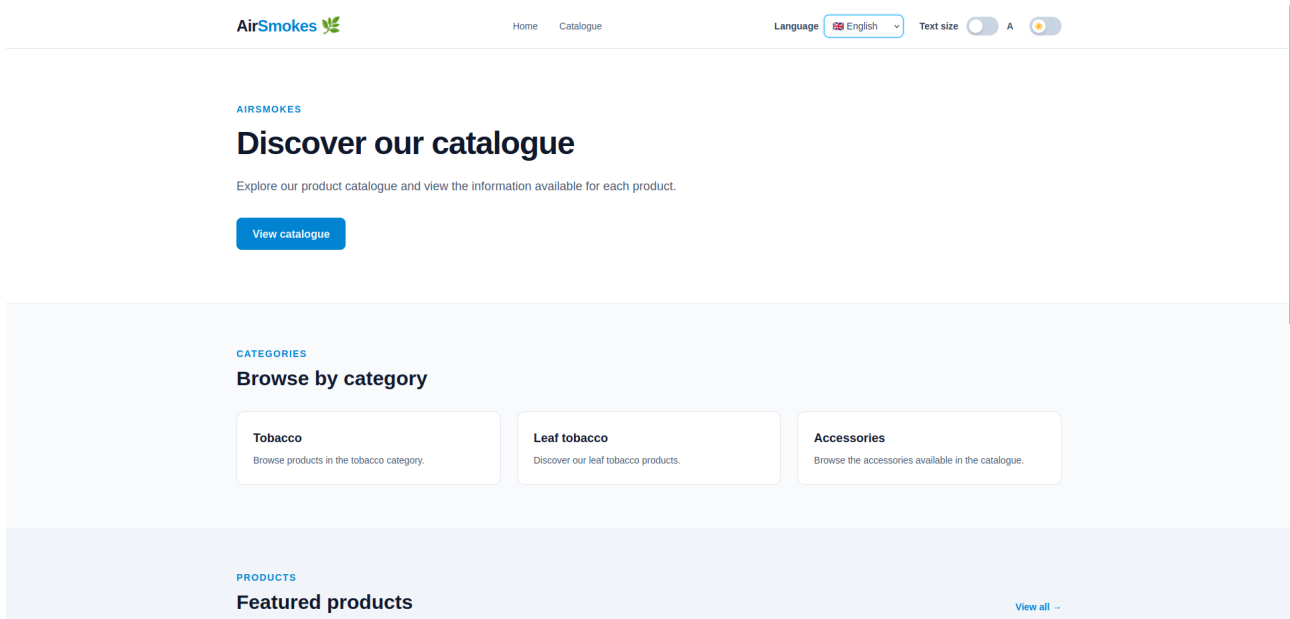


FIGURE 12 – Interface en anglais

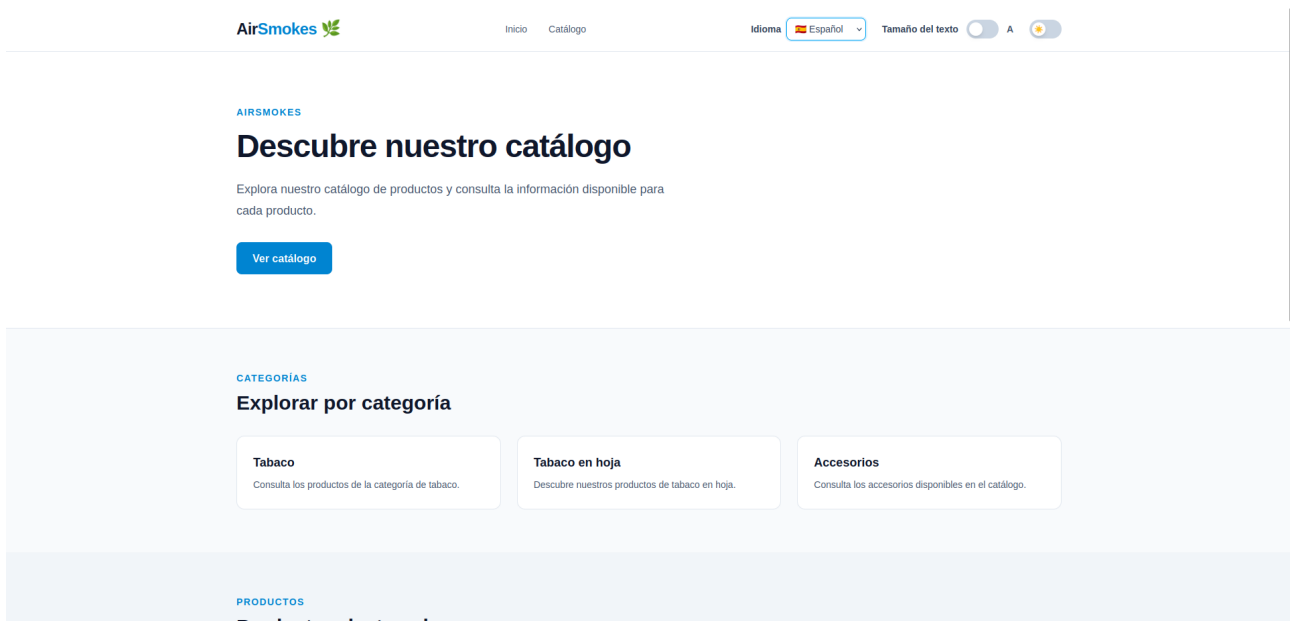


FIGURE 13 – Interface en espagnol

4.3.4 Modifications et ajouts au code source

La fonctionnalité repose sur un état permettant de déterminer la langue actuellement sélectionnée. Les différents textes affichés dans l'interface sont ensuite adaptés en fonction de cette langue.

Le changement de langue est effectué à partir du contrôle prévu à cet effet et permet de mettre à jour les textes affichés sans avoir à modifier manuellement le contenu de chaque composant.

La langue sélectionnée est également conservée dans le stockage local du navigateur afin de maintenir la préférence de l'utilisateur lors de ses prochaines visites.

1 | `"use client";`

```

2
3 import { useLanguage } from "../LanguageProvider";
4
5 const languages = [
6   {
7     code: "fr",
8     label: "Français",
9     flag: "🇫🇷",
10  },
11  {
12    code: "en",
13    label: "English",
14    flag: "🇺🇸",
15  },
16  {
17    code: "es",
18    label: "Español",
19    flag: "🇪🇸",
20  },
21 ] as const;
22
23 export default function LanguageToggle() {
24   const { language, setLanguage, t } = useLanguage();
25
26   function handleLanguageChange(value: string) {
27     if (value === "fr" || value === "en" || value === "es") {
28       setLanguage(value);
29     }
30   }
31
32   return (
33     <div className="flex items-center gap-2"> <label
34       htmlFor="language"
35       className="text-sm font-semibold text-slate-700 dark:text-slate-300"
36     >
37       {t.header.language} </label>
38       ““
39       <select
40         id="language"
41         value={language}
42         onChange={(event) => handleLanguageChange(event.target.value)}
43         aria-label={t.header.language}
44         className="rounded-lg border border-slate-300 bg-white px-3 py-2 text-
45           sm font-medium text-slate-700 outline-none transition focus:
46             border-sky-500 focus:ring-2 focus:ring-sky-100 dark:border-slate-
47               700 dark:bg-slate-900 dark:text-slate-200 dark:focus:border-sky-
48                 500 dark:focus:ring-sky-950"
49       >
50         {languages.map((item) => (
51           <option key={item.code} value={item.code}>
52             {item.flag} {item.label}
53           </option>
54         ))}
55       </select>
56     </div>
57   );
58 }

```



```

51     </select>
52 </div>
53   ""
54
55 );
56 }

```

```

1  "use client";
2
3  import { createContext, useContext, useState, type ReactNode } from "
    react";
4
5  import { translations, type Language } from "@app/i18n/translation";
6
7  type LanguageContextType = {
8    language: Language;
9    setLanguage: (language: Language) => void;
10   t: (typeof translations)[Language];
11 };
12
13 const LanguageContext = createContext<LanguageContextType | undefined>(
14   undefined,
15 );
16
17 export function LanguageProvider({ children }: { children: ReactNode }) {
18   const [language, setLanguageState] = useState<Language>(() => {
19     if (typeof window === "undefined") {
20       return "fr";
21     }
22
23     ""
24     const savedLanguage = localStorage.getItem("language");
25
26     if (
27       savedLanguage === "fr" ||
28       savedLanguage === "en" ||
29       savedLanguage === "es"
30     ) {
31       return savedLanguage;
32     }
33
34     return "fr";
35     ""
36   });
37
38
39   function setLanguage(newLanguage: Language) {
40     setLanguageState(newLanguage);
41     localStorage.setItem("language", newLanguage);
42   }
43
44   return (
45     <LanguageContext.Provider

```

```

46 | value={{
47 |   language,
48 |   setLanguage,
49 |   t: translations[language],
50 | }}
51 | >
52 | {children}
53 | </LanguageContext.Provider>
54 | );
55 | }
56 |
57 | export function useLanguage() {
58 |   const context = useContext(LanguageContext);
59 |
60 |   if (!context) {
61 |     throw new Error("useLanguage must be used inside a LanguageProvider");
62 |   }
63 |
64 |   return context;
65 | }

```

5 Conclusion

Le projet AirSmokes permet de mettre en place les principales fonctionnalités prévues pour le MVP, notamment la consultation du catalogue, la recherche dynamique, le filtrage par catégorie et la consultation des détails d'un produit. Des fonctionnalités d'inclusivité et d'accessibilité ont également été ajoutées afin d'améliorer l'expérience des utilisateurs. Il est notamment possible de modifier la taille du texte, de choisir entre un thème clair et un thème sombre et de changer la langue de l'interface. Le projet constitue actuellement un catalogue fonctionnel, mais certaines fonctionnalités d'un véritable commerce en ligne restent à développer, notamment la gestion d'une base de données et la possibilité de passer de réelles commandes.